

BSc (Hons) Computer Science with Software Engineering
Coventry University delivered at NIBM

NB6007CEM - Web API Development

Collections



Day 2 (AM) - Session 3

Niranga Dharmaratna | niranga@nibm.lk

S3

*Collections
& URI Design*

Session 3 - What Today Delivers



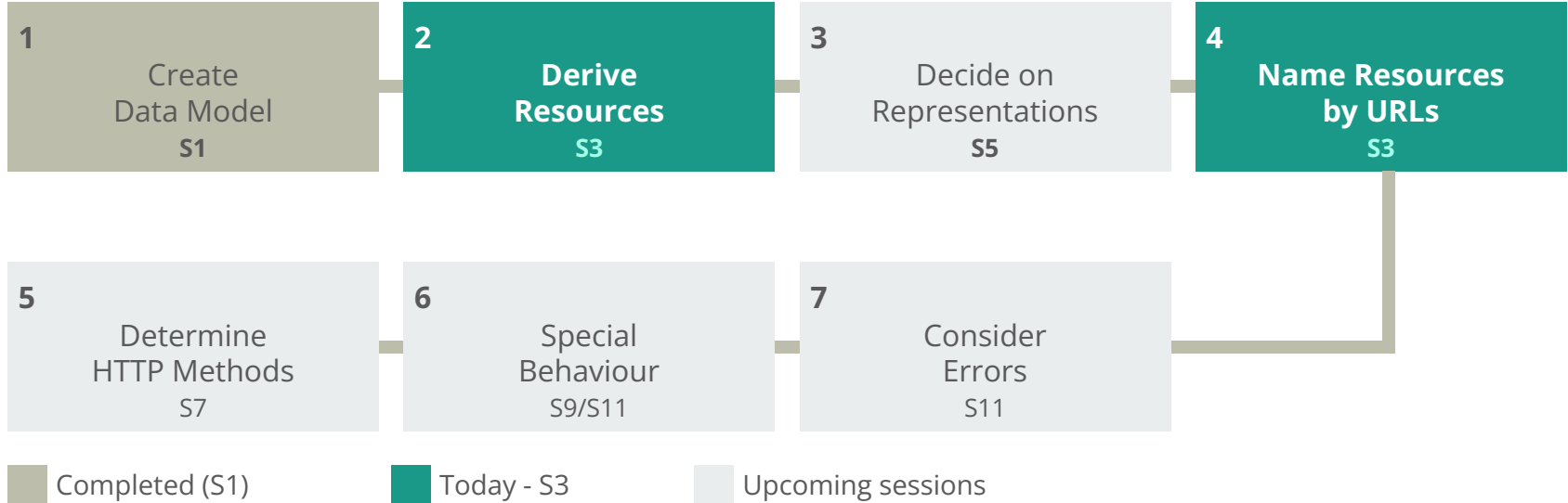
From data model to named resources - giving the API its shape

By the end of this session you will be able to:

- Distinguish atomic from collection resources using the §4.1 and §4.2 derivation rules
- Apply the §4.6 scoping test to decide when a collection is first-class versus nested under a parent
- Name resources following all five §5.1 naming rules simultaneously
- Produce and evaluate the complete resource map for the tuk-tuk system

Design guidelines covered: §4.1, §4.2, §4.6, §5.1, §5.6

The REST API Design Pipeline



A - From Data Model to Resources

From Data Model to Resource Model



A data model and a resource model are not the same thing

- The data model specifies entity properties in an abstract, implementation-independent form - no format or transport decisions are made while the data model is being defined
- The resource model specifies what clients interact with - it is shaped by how clients use the API, not by the ER diagram alone
- Consequence: new resources may be derived that have no entity type in the ER diagram; some ER entities may not become independent top-level resources

The guiding principle:

- Clients win over data - client interaction requirements drive the resource model; the data model drives the implementation

Five Resource Types - §4 Taxonomy

The design guidelines (§4) define five resource types that may be derived from a data model. Not all are derived in every API - only those warranted by client needs.

Type	What it represents	Derived from	When
Atomic (§4.1)	An entity exchanged as a complete whole	Entity types	TODAY - S3
Collection (§4.2)	A grouped set of atomics of the same type	Entity types (grouped)	TODAY - S3
Composite (§4.3)	An aggregate of multiple entities manipulated as a whole	Groups of related entities	S5
Processing Fn (§4.5)	A computation, function, or partial update	Functional requirements	S5
Controller (§4.4)	A multi-resource atomic operation for data consistency	Integrity requirements	S5

S3 covers types 1 and 2. Types 3, 4, and 5 are introduced in S5.

Atomic Resources - §4.1



Definition (§4.1): entities of the data model that are exchanged as a whole via the API become atomic resources

Three conditions - all must hold for an entity to qualify:

- **Accessed in multiple client scenarios - its complete set of details is needed broadly**
- **Manipulating it does NOT require traversal of relationships to other entities**
- **Retrieved, created, or deleted as a complete unit - never as a partial fragment**

ANALOGY

A NIC (National Identity Card) is exchanged as a complete unit. You never request "just the birthday" or "just the address" from the NIC separately - you receive the whole card. Province, District, PoliceStation, and Vehicle work the same way: the API exchanges each as a whole record, not as fragments. Limit: a real NIC is immutable once issued; atomic resources can be updated with PUT.

Generator Mistake - Standalone LocationPing

GENERATOR PRODUCES:

/pings/{ping-id}

Apply the §4.1 independence test to LocationPing:

- **Condition 1 - multiple client scenarios? No. Police never request "ping 4827" in isolation. Every ping request names a vehicle first.**
- **Condition 2 - independent of relationships? No. A ping has no context without its vehicle. Traversal through Vehicle is always required.**
- **Condition 3 - exchanged as a complete whole? No basis exists. No client ever needs an isolated ping record.**

Result: LocationPing fails all three conditions. It is NOT an atomic resource.

§4.6 formalises this: "if manipulating instances requires traversal of associated relationships, the entity is not an atomic candidate". LocationPing belongs to a scoped collection under Vehicle - introduced in Section B.

Atomic Resources in the Tuk-Tuk System



Applying the §4.1 independence test to each domain entity:

- Province - accessed directly (name, code, HQ); no traversal needed. Passes all three conditions.
- District - accessed directly (name, province ref, district seat); the province ref is a property, not a traversal. Passes.
- PoliceStation - accessed directly (name, district, OIC contact). Passes.
- Vehicle - accessed directly (plate number, driver, device_id, active status). Passes.
- User - accessed directly (name, role, jurisdiction scope). Passes.

Note: the User collection endpoint (/users) is deferred to S7. The auth layer determines who may create users and what scope they carry. The atomic exists; the collection factory arrives with authentication.

Collection Resources - §4.2



Definition (§4.2): atomic resources of the same type, grouped into a set, become a collection resource

The collection type does NOT appear in the ER diagram - it is derived from client need

Two derivation triggers - either is sufficient:

- **Catalogue:** clients need to browse or filter instances of that entity type
- **Creation:** the API supports creating new instances of that entity type

Factory principle (§7.3): POST to the collection creates a new member

ANALOGY

The Motor Traffic Department maintains a vehicle register - a browsable ledger of all registered vehicles in a district. The register is the collection resource; each vehicle certificate is an atomic member. You go to the register to browse, search, and add records. The register exists independently of any individual certificate. Limit: the real DMT register is owned by one authority per district; in the API, a collection resource is a single canonical set accessible to all permitted clients.

Generator Mistakes - Two Collection Traps



TRAP 1 - Wrong singular name

- Generator produces /province, /vehicle, /station (singular nouns for collections)
- §5.1 rule: "names of collections should be pluralized" - the singular is ambiguous between the collection and one member
- Correct: /provinces, /vehicles, /stations, /districts

TRAP 2 - Phantom collection (invented, not derived)

- Generator produces /ping-history as a top-level route
- This entity type does not appear as a first-class type in the domain model - there is no "PingHistory" entity, only a scoped set of LocationPings under a vehicle
- No catalogue or creation trigger exists at the top level for ping-history
- Correct: LocationPing will be derived as a scoped collection under Vehicle - /vehicles/{vehicle-id}/pings (Section B)

Collection Resources in the Tuk-Tuk System



Applying both derivation triggers to each atomic entity type:

- **/provinces** - catalogue: police browse all provinces; creation: register a new province. Both triggers. Collection warranted.
- **/districts** - catalogue: police browse all districts; creation: register a new district. Both triggers. Collection warranted.
- **/stations** - catalogue: police browse all stations; creation: register a new station. Both triggers. Collection warranted.
- **/vehicles** - catalogue: police browse all vehicles; creation: register a new vehicle. Both triggers. Collection warranted.
- **LocationPing**: NOT a first-class collection. No catalogue trigger exists at the top level (police never browse all pings globally). Derivation as a scoped collection is addressed in Section B.

All URIs listed here are paths only. HTTP methods are determined in S7.

B - Relationships and Scoping

Interpreting Relationships - §4.6



§4.6: how to interpret ER relationships when deriving the resource model

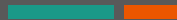
- **Independence test:** "if manipulating instances does NOT require traversal of relationships, the entity is an atomic candidate"
- **Scoping test:** if a collection only makes sense under a specific parent, it is a scoped collection

Two categories result:

- **First-class:** the global set is meaningful to clients - stands at the root level
- **Scoped:** meaningful only under a specific parent - nested in the path

ANALOGY

When you request electricity meter readings, you request readings for meter number 4512 - not "all readings from all meters in the Western Province." The readings only exist, and only make sense, under that specific meter. `/vehicles/{vehicle-id}/pings` works the same way: the pings are always scoped to one vehicle. Limit: a meter belongs to one utility; a tuk-tuk can cross district boundaries. The scoping analogy holds at the API path level, not at the physical ownership level.



§4.6 - SCOPING RULE

The scoping decision is a client need, not a data model relationship.

The ER diagram shows that Vehicle has many LocationPings. But the API is not the ER diagram. The question is: do clients ever need all pings globally? When the answer is no, the collection is scoped - and the URL path makes that constraint permanent and visible to every caller.

Scoped vs Unscoped - The Tuk-Tuk Decision

FIRST-CLASS (unscoped)

Globally meaningful - police DO query these at the top level:

- **/provinces**

Police need: "show all provinces in the system"

- **/districts**

Police need: "show all districts (filter by province)"

- **/stations**
- **/vehicles**

Client evidence: police need to browse all vehicles in the system, filter by area, or register a new one.

SCOPED (under a parent)

Meaningful only under a specific parent:

- **/vehicles/{vehicle-id}/pings**

Police need: "show pings for vehicle WP-CAB-1234"

Client evidence: police NEVER request "all pings in Sri Lanka". Every ping request names a specific vehicle first.

The ER relationship (Vehicle 1-* LocationPing) becomes a scoped URI path, not a top-level collection.

Generator Mistake - Global /pings Collection

GENERATOR PRODUCES:

/pings?vehicleId=WP-CAB-1234

Three reasons §5.6 rejects the top-level /pings design:

- Client-need test fails: "the set of all pings globally is never a meaningful resource." The first-class collection criterion (§4.6) requires that the complete global set be of interest to clients. It is not.
- §5.6 requires path-level scoping when a collection is relationship-dependent. When a collection only makes sense under a specific parent instance, the parent must appear in the path - not as a query parameter.
- ?vehicleId= is a §5.7 query-string parameter for FILTERING an existing collection. You cannot use a query string to scope a collection that does not exist at the root. Filtering and scoping are different concerns.

Correct: /vehicles/{vehicle-id}/pings - the vehicle-id in the path makes the parent relationship explicit and permanent for every caller.

Pagination and filtering (§9, §10.2-10.4) use query strings on existing collections - they do not substitute for path-level scoping.

C - Naming the Resources

URI Naming Rules - §5.1



§5.1: "proper naming is key for an API to be understandable by clients." Five rules that must all hold simultaneously:

- **Rule 1 - Nouns not verbs: resources are things, not actions. Processing and controller resource types use verbs (covered in §5). Today, all resources are nouns.**
- **Rule 2 - Lowercase only: case sensitivity in URIs causes ambiguity between clients and servers. Lowercase removes all ambiguity.**
- **Rule 3 - Hyphens, not underscores or camelCase: underscores disappear under hyperlinks; camelCase is a programming convention, not a URI convention.**
- **Rule 4 - Plural for collections, singular for atomic members: /vehicles names the set; /vehicles/{vehicle-id} names one member.**
- **Rule 5 - Forward slash for hierarchy: parent immediately precedes child. No trailing slash.**

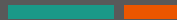
Analogy: Sri Lanka Post address format - one standard, no exceptions. Detail in session notes.

Generator Mistakes - §5.1 Violations

Every path below was produced by a generator for the tuk-tuk system. Each violates at least one §5.1 rule.

Generator produces	§5.1 rule broken	Correct version
/getVehicleList	Verb-as-noun; camelCase (Rules 1 + 3)	/vehicles
/Vehicle_Location	Uppercase; underscore (Rules 2 + 3)	/vehicle-locations
/vehicleLocation	camelCase (Rule 3)	/vehicle-locations
/locationPings	camelCase + unscoped - missing parent (Rules 3 + §5.6)	/vehicles/{vehicle-id}/pings
/vehicles/id/pings	Literal "id" not a template variable (§5.6)	/vehicles/{vehicle-id}/pings

Use the URI Linter demo (Activity slide) to check paths from your own generated routes.



§5.1 - NAMING RULE

A URI is a name, not a sentence. The API speaks in nouns, not instructions.

When a generator produces `/getVehicleLocation`, it is writing a method call - not naming a resource. The resource already exists; it needs a name clients can find and remember. `/vehicles/{vehicle-id}` is that name. The HTTP method (GET, POST, etc.) handles the verb - so the URI never needs one.

URI Templates and Scoped Collections - §5.6



Definition (§5.6, RFC 6570): a URI template contains curly-bracket variables substituted by values when the template is used by a client

- **Pattern:** `/ {collection-name} / {member-id}` - the member-id immediately follows the collection name and uniquely identifies one atomic member
- **Scoped extension:** `/ {collection} / {member-id} / {scoped-collection}` - adds a child collection under the specific parent identified by member-id
- **Template vs literal:** `{vehicle-id}` is a variable substituted on every call; `vehicle-id` without braces is a literal string that routes to nothing
- **Variable naming:** entity-type-name followed by -id, all lowercase-hyphen. Depth rule: only nest as deep as the client relationship requires.

Analogy: police stop-and-check form blank - "Vehicle Reg. No.: ____". The template is the printed blank; the call fills it in. Detail in session notes.

The Tuk-Tuk Resource Map - Session 3

Everything derived in S3. No HTTP methods, no JSON, no authentication - those belong to later sessions.

UNSCOPED COLLECTIONS + MEMBERS

/provinces

/provinces/{province-id}

/districts

/districts/{district-id}

/stations

/stations/{station-id}

/vehicles

/vehicles/{vehicle-id}

SCOPED COLLECTION

/vehicles/{vehicle-id}/pings

LocationPings - always under a specific vehicle

NOT YET DERIVED

HTTP methods

Session 7

JSON representations

Session 5

Composite resources, /users collection

Session 5 / Session 7

Activity - URI Naming Linter

- Open the URI Linter demo on your device
- Type each of these paths and read the violations flagged:

/getVehicleLocation

/Vehicle_Location

/vehicleLocation

- Type a path from your own domain model
- For each violation: name the §5.1 rule it breaks - in your own words, not the linter label
- Find one path that passes clean

Pairing:

Work in pairs. Both must explain each violation by rule name - not just flag it. The S4 micro-viva checks you individually.

Coursework connection:

The §5.1 critique of your generated routes is scored under API Design. You will commit your first named routes in S4 - every path in your repo will be evaluated against these five rules.

Discussion

User is listed as an atomic resource - but /users is not on the resource map. Which §4 rule explains the gap? Which session introduces the /users collection, and why does auth determine the answer?

Think through your answer before S4. You may be asked at the micro-viva.

Questions & Discussion



S3 ends here - S4 builds this afternoon

S3 Complete

Day 2 (PM)
Session 4 builds
the route layer

Disclaimer & Copyrights



The information presented in this presentation is intended for general informational purposes only. It is not meant to be comprehensive or to constitute professional advice. Any reliance you place on such information is strictly at your own risk. While I strive to keep the information accurate and up-to-date, I make no representations or warranties of any kind, express or implied, about the completeness, accuracy, reliability, suitability, or availability concerning the content within this presentation.

This presentation may include references or links to third-party websites or content. I do not endorse the views expressed or the information provided on such external sites. I am not responsible for the content of any linked site or any link contained in a linked site.

Any action you take upon the information presented in this presentation is strictly at your own discretion. I will not be liable for any losses or damages in connection with the use of this information.

This disclaimer is subject to change without notice.

@ Images and artworks used in this presentation are used for general informational purposes only. It may contain copyrighted material, the use of which may not have been specifically authorised by the copyright owner. This material is available in an effort to explain issues relevant to the topic. This should constitute a 'fair use' of any such copyrighted material.

Last modified: 14 Jun 2026